Optimize Against EA

API Documentation

Developer Reference

GameLab 2 — Julius-Maximilians-Universität Würzburg

Philipp Sehne
Philipp.Sehne@stud-mail.uni-wuerzburg.de

David Ahrens
David.Ahrens@stud-mail.uni-wuerzburg.de

July 16, 2026

Contents

Chapter 1

Introduction

1.1 Scope

Optimize Against EA is a purely client-side React 19 + TypeScript single-page application (built with Vite). It exposes no HTTP/REST endpoints; "API" in this document therefore means the **programmatic interfaces of the codebase**: the exported types, engine functions, Web Worker message protocols, React hooks, and data serialization formats a developer needs in order to extend the application or reuse its parts.

This document complements the *Handbook & User Manual*, which covers the application from the player's perspective.

1.2 How to read a reference entry

Each documented symbol appears as an entry with its source file in the header, followed by its signature, a description, and where relevant **Params**, **Returns**, **Side effects** and **Notes** fields. All paths are relative to `OptimizeAgainstEA/OptimizeAgainstEAApp/src/`.

1.3 Conventions used in the codebase

- Type-only imports use `import type {...}`.
- React components are `function` declarations; engine and utility functions are `const` arrow functions or plain functions.
- No `any`; all callback parameters are explicitly typed.
- Fitness in the map/maze EAs is *minimized* (0 = optimal); fitness in the shooter GA is *maximized*.
- Randomness is injected as `RNG = () => number` (a `Math.random`-compatible function) so engines stay deterministic under a seeded RNG.

Chapter 2

Architecture Overview

2.1 Layering

Every game module follows the same four-layer structure. Lower layers never import from higher ones:

Layer	Contents
`types/`	Pure type contracts and default configurations. No logic.
`engine/`	Pure, React-free logic: problem construction, EA operators, codecs, geometry. Unit-testable in isolation.
`engine/*.worker.ts`	Web Workers running the EA off the main thread; communicate via typed message unions (`WorkerInMessage`/`WorkerOutMessage`).
`hooks/`	React hooks bridging engine/workers to component state.
`components/`	Presentational and interactive React components.

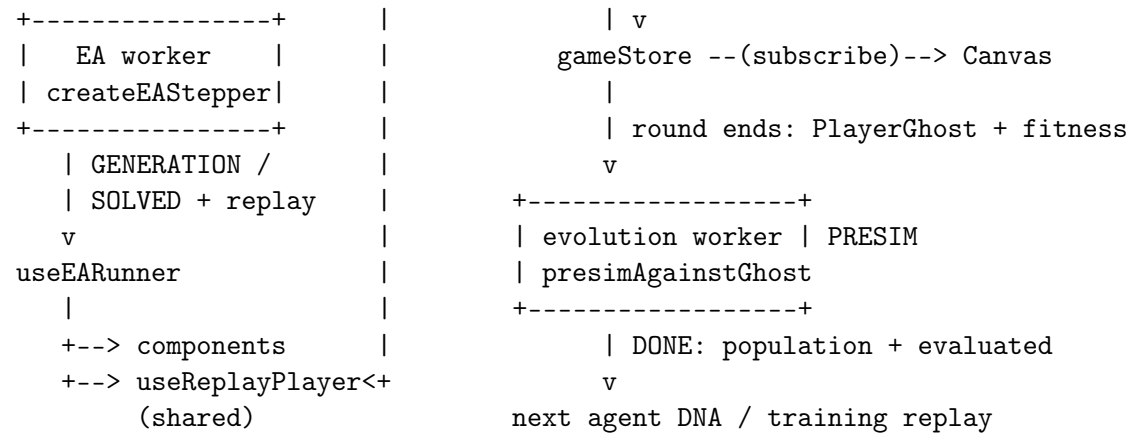
2.2 Module map

Module	Route	Chapter
`modules/BattleShips/`	`/PeakFinder`	4
`modules/mazeGame/`	`/MazeExplorer`	5
`modules/shooterGame/`	`/ShooterGame`, `/HordeGame`	6
`components/, hooks/, context/`	— (shared)	3

2.3 Data flow

The three modules solve the same problem in two different shapes. Peak Finder and Maze push their EA into a **worker** and pull generations back as messages; the shooter keeps its simulation on the main thread at 60 Hz and pushes state through an **observable store**, moving only the heavy presimulation off-thread.

```
Peak Finder / Maze                  Shooter
------------------                  -------
share code                          ShooterSettings
   | decodeProblem                     | makeInitialGameState
   v                                   v
ProblemInstance --------+           GameState <---- input (ref, no re-render)
   | ProblemSource      |              ^ |
   v  (postMessage)     |              | | update(state, dt, input, ...)
```



Two invariants hold across both shapes. Every EA is *deterministic in its inputs* — the map/maze workers rebuild the identical problem from a serializable `ProblemSource` or seed, and the shooter's ghost simulation seeds its RNG from the DNA itself — so what a replay shows is exactly what the algorithm scored. And no engine imports React: the worker or store boundary is where a pure simulation meets the UI.

Chapter 3

Shared Infrastructure

3.1 Global state and settings

`SettingsContext / SettingsProvider` — `context/SettingsContext.tsx`

Global settings provider; wraps all routes in `App.tsx`. Holds three independent settings objects, each with its own setter:

```
const { eaSettings,      setEaSettings,      // EASettings
        shooterSettings, setShooterSettings, // ShooterSettings
        hordeSettings,   setHordeSettings,   // HordeSettings
      } = useSettings(); // throws outside a <SettingsProvider>
```

`EASettings` tunes the Solo-Play shooter GA (mutation rate/strength, presim generations, population size, crossover type, hall of fame, analytics retention, injection deviation). `ShooterSettings` holds starter DNA, round duration, tug-of-war win threshold, player stats and the powerup-choice options. `HordeSettings` is deliberately separate from `EASettings` (own difficulty presets, own starter DNA padded with neutral 0.5s for the Horde-only genes, wave sizing, map id, custom map layout, kills-per-upgrade).

Notes. State is React-only — *not* persisted; a reload restores the `default*Settings`. `resetEASettings()` / `resetShooterSettings()` / `resetHordeSettings()` return fresh defaults (the shooter/horde variants reset the starter DNA to all-0.1 "untrained" genes).

3.2 Layout components

`GameLayout, LAYOUT` — `components/layout/GameLayout.tsx`

Three-column page shell (`leftBar | canvas | sidebar`) used by the shooter pages. Exports the `LAYOUT` constant (column widths, spacing, panel colours) for canvas-sizing arithmetic.

Notes. Pass the combined sidebar width to `useScaledCanvas` so the canvas scales correctly.

`PageContainer` — `components/layout/PageContainer.tsx`

`PageContainer({ children, backgroundImage? })` — full-viewport page wrapper (`.page-container`); `backgroundImage` is a URL or imported image rendered as a covering, centered background. Default export.

3.3 Shared hooks

useScaledCanvas (worked example) — hooks/useScaledCanvas.ts

```
const { containerRef, scale } = useScaledCanvas({
  baseWidth:    number,   // logical canvas width  (e.g. ARENA.WIDTH)
  baseHeight:   number,   // logical canvas height
  sidebarWidth: number,   // total horizontal space taken by side panels
  padding:      number,   // default 16
});
```

Computes the largest uniform scale at which the logical canvas fits the measured container minus `sidebarWidth` and `padding`, preserving the aspect ratio.

Returns. `containerRef` (attach to the element whose size to measure — a `ResizeObserver` keeps `scale` current) and the numeric `scale` factor to apply to the canvas.

useViewport — hooks/useViewport.ts

`useViewport(): { W, H }` — the current `window.innerWidth`/`innerHeight`, re-rendered on every window `resize`.

useOrientationLock — hooks/useOrientationLock.ts

`useOrientationLock(type = 'landscape')` — locks the screen orientation while mounted, unlocking on unmount. Only attempts the lock when fullscreen is active (a platform requirement) and re-locks on every `fullscreenchange`. Fails silently where unsupported (iOS Safari); works on Android Chrome in fullscreen.

useReplayPlayer — hooks/useReplayPlayer.ts

```
const { frameIndex, currentFrame, totalFrames, isPlaying,
        isFirst, isLast, next, prev, goTo, play, pause } =
  useReplayPlayer<Frame>(frames, autoplayIntervalMs = 1200);
```

Playback state machine for any recorded frame list — shared by the PeakFinder EA replay, the Maze EA replay and the generation replay. Generic over the frame type: it only navigates, never looks inside a frame. `play(ms?)` auto-advances at the given dwell (default `autoplayIntervalMs`) and self-terminates on the last frame; `next`/`prev`/`goTo` clamp to the valid range.

useSavedLibrary — hooks/useSavedLibrary.ts

```
const { entries, save, remove, rename } =
  useSavedLibrary<Input, Entry extends SavedEntryBase>(storageKey, toEntry);
interface SavedEntryBase { id: string; name?: string; createdAt: number }
```

localStorage-persisted library of player creations — the shared core behind `useSavedMaps` (`bs.savedMaps`) and `useSavedMazes` (`maze.savedMazes`). `toEntry` converts the game's input into an entry; its `id` de-duplicates (saving an existing id is a no-op, and `save` returns the id either way). A blank name in `save`/`rename` clears the name. Storage access is guarded, so private-mode failures degrade to an in-memory list.

3.4 Hint system

Page-wide, toggleable contextual help (`components/hints/`). Content lives exclusively in `hintContent.ts` (BattleShips) and `mazeGame/hints/mazeHintContent.ts`.

useHints (worked example) components/hints/HintContext.tsx

```
const { showHint, dismiss } = useHints();
showHint('vsEa.afterProbe', { vars: { gens: '5' }, actions: [...] });
```

Fires a hint by id. No-ops when hints are disabled or the hint is marked `once` and was already seen this session.

Side effects. Persists `enabled` to `localStorage` (`app.hints.enabled`) and `seen` ids to `sessionStorage` (`app.hints.seen`) — so an on/off choice survives across visits while `once` hints re-arm in a new browser session.

HintDef components/hints/hintContent.ts

```
{ title?, body, style: 'modal' | 'toast', once?, pauses? }
```

`pauses`: `true` blocks interaction behind a backdrop until dismissed.

HintPopover components/hints/HintPopover.tsx

Anchored, non-blocking coach-mark: wrap any element to pin a hint bubble to it.

```
<HintPopover id="vsEa.replayButton" placement="top" show={canReplay}>
  <button>Watch Last Replay</button>
</HintPopover>
```

Props: `id` (a `HintId` from `hintContent.ts` — the wording lives there), `children` (the anchor), `placement` (`'top' | 'top-start' | 'top-end' | 'bottom' | 'left' | 'right'`, default `'right'`), `show` (extra condition on top of enabled/once-seen, default `true`), `vars` (fills `{placeholders}` in the body), `actions` (buttons; falls back to a single "Got it"), `dismissAfter` (auto-dismiss in ms). The bubble renders via a portal to `document.body` and is clamped to stay fully on-screen.

Notes. CSS-grid caveat: the wrapper is `display: inline-block`, so inside a grid it would itself become the grid item — wrap the child *inside* the grid item instead.

3.5 Help system

HELP_TOPICS / HelpButton components/help/helpContent.tsx

Content registry for the site-wide help modals. Adding a topic = one entry in `HELP_TOPICS` plus a `<HelpButton topic="..." />`. Each topic provides a `Gameplay` and a `Technical` tab component.

3.6 Firebase

firebase lib/firebase.ts

Exports `firebaseApp` (the initialized Firebase app) and `db` (its Firestore handle). The client-side web config is hardcoded in this file (project `optimizeagainstea`); as a public web API key it is not a secret — access control lives in the Firestore security rules.

Notes. The only consumer is the community Raidboss mode (`modules/shooterGame/game/raidbossStore.ts`); every other feature is fully client-local. Swapping the backing project means editing this one config object.

3.7 Utilities

rng — utils/rng.ts

```
type RNG = () => number;          // next value in [0, 1)
makeLCG(seed?: number): RNG
```

Seeded linear congruential generator (`s = 1664525 s + 1013904223` mod 2^{32}, via `Math.imul` — pure integer math, so the stream is identical on every device). The single random source of the PeakFinder and Maze EAs and of maze generation: same seed ⇒ fully reproducible run. Without a seed, a fresh `Math.random()`-derived one is used; the degenerate all-zero state is remapped.

listenable — utils/listenable.ts

```
createListenable(): { notify(): void;
                      subscribe(fn: () => void): () => void }
```

The observer plumbing behind every module-level singleton store (`gameStore`, `analyticsStore`, `hordeGameStore`, …): spread the result into a store object literal; domain methods mutate their fields and call `this.notify()`. `subscribe` returns the matching unsubscribe function, so components can hook in directly or via `useSyncExternalStore`.

Chapter 4

Peak Finder Module (BattleShips)

All paths relative to `modules/BattleShips/`.

4.1 Type contracts (`types/`)

`ProblemInstance (worked example)` — `types/map.ts`

The central abstraction: game modes only ever hold a `ProblemInstance`, never a raw `MapConfig`.

```
interface ProblemInstance {
  evaluate: (x: number, y: number) => number; // 0 = global min, 1 = far away
  bounds:   { xMin: number; xMax: number; yMin: number; yMax: number };
  isWin:    (x: number, y: number) => boolean;
  displayExponent?: number; // display-only heatmap colour curve; see Notes
  metadata?: { name?: string;
               description?: string;
               globalMinimum?: { x: number; y: number; value: number };
               winRadius?: number }; // extent of the win zone, for overlays
}
```

Notes. Implemented by `createMapProblem` (`engine/functionSurface.ts`) and `createFunctionProblem` (`engine/functionProblem.ts`). New problem sources only need to produce this shape. `evaluate` is the single source of truth — the player's reading, the EA's fitness and the heatmap colour all come from it — so a problem must return a value that already spans $[0, 1]$ sensibly; do any shaping in the surface engine, never as a display curve. `displayExponent` is the one purely-cosmetic exception: when set, the heatmap colours by $\texttt{evaluate}^{\texttt{displayExponent}}$ instead of `evaluate` (default 1 = untouched), without ever affecting readings, wins or the EA. Surface maps leave it unset; benchmark functions set it to 0.55 to keep more colour near the optimum.

`Coordinate, Minimum, MapConfig, ProbeResult` — `types/map.ts`

```ts
interface Coordinate { x: number; y: number }  // normalized 0-1
interface Minimum {
  id: string;
  position: Coordinate;
  isGlobal: boolean;  // exactly one minimum per map is the global one
  floor?: number;     // depth of a local minimum, as a fraction of basinScale
}
interface MapConfig {
  id: string;               // 6-char uppercase share id
  minima: Minimum[];
  bounds: { xMin: number; xMax: number; yMin: number; yMax: number };
  winRadius: number;        // normalized radius around the global minimum
  basinScale?: number;      // basin width (normalized); see Notes
  createdAt: number;        // epoch ms
}
interface ProbeResult { position: Coordinate; value: number; isWin: boolean }
```

Notes. `Minimum.floor` is the local minimum's depth *as a fraction of the map's* ***basinScale*** (not an absolute value), so a decoy is equally deceptive on a Small map (wide basins) and a Huge one (narrow basins) — lower = deeper, more deceptive. When omitted, a position-seeded pseudo-random fraction in $[\texttt{LOCAL\_MIN\_FLOOR\_MIN}, \texttt{LOCAL\_MIN\_FLOOR\_MAX}] = [0.18, 0.55]$ is used, so the same map code always yields the same floors. The floor is ignored (always 0) for the global minimum. `basinScale` is how wide one minimum's basin is in normalized units; it comes from the size preset (`MAP_SIZES`) and fixes a basin's size to the map rather than to how many minima share it. Legacy codes without it fall back to the Medium preset's scale. `bounds` is always the unit square today.

`EAConfig (worked example)` — `types/ea.ts`

Complete configuration of the Vs-EA opponent. `DEFAULT_EA_CONFIG` provides the defaults (identical to the `'normal'` preset); `EA_PRESETS` bundles named presets for the UI.

Field	Default	Meaning
`populationSize`	30	Individuals per generation
`maxGenerations`	200	Hard stop
`crossoverRate`	0.7	Probability of recombination
`mutationRate`	0.3	Probability of mutating a child
`mutationStrength`	0.25	Max. perturbation (normalized coords)
`mutationDecay`	0.90	Per-generation strength multiplier
`winPopulationFraction`	0.35	Fraction inside win radius ⇒ solved
`selectionStrategy`	`'elitist'`	`| 'tournament' | 'roulette'`
`crossoverStrategy`	`'uniform'`	`| 'arithmetic' | 'singlePoint'`
`mutationStrategy`	`'gaussian'`	`| 'uniform' | 'cauchy'`

`Individual, Generation, SelectionFn, CrossoverFn, MutationFn, RNG`
`types/ea.ts`

```ts
interface Individual {
  position:   Coordinate;
  fitness:    number;  // problem.evaluate(position) - minimized, 0 = summit
  isSolution: boolean; // inside the win radius
}
interface Generation {
  index:       number;
  individuals: Individual[]; // sorted best-first
  best:        Individual;
  meanFitness: number;
}
type RNG         = () => number; // Math.random-compatible
type SelectionFn = (population: Individual[], rng: RNG) => Individual;
type CrossoverFn = (a: Individual, b: Individual, rng: RNG) => Coordinate;
type MutationFn  = (coord: Coordinate, generationIndex: number,
                    config: EAConfig, rng: RNG) => Coordinate;
```

Notes. Operator implementations are looked up from the strategy registries in `engine/ea/operators.ts`. Mutation receives the generation index so it can apply the decayed strength `mutationStrength` · `mutationDecay`^{gen}.

4.2 Engine (`engine/`)

`createMapProblem` — `engine/functionSurface.ts`

`createMapProblem(config: MapConfig): ProblemInstance` — builds the smooth surface from the placed minima. First a raw cone surface $s(x,y) = \text{smoothMin}_{m\,\in\,\text{minima}}\big(d((x,y),m) + \mathit{floor}_m\big)$ blends the per-minimum cones with a polynomial smooth-minimum (Quilez) so the terrain has no creases along Voronoi seams; floor_m is 0 for the global minimum and the explicit or seeded fractional floor (times `basinScale`) for locals. That raw distance is then mapped to $[0,1)$ against the map's own `basinScale` by exponential saturation:

$$v(x,y) \;=\; 1 - 2^{-\,b(x,y)^{\texttt{FAR\_FALLOFF}}}, \qquad b(x,y) \;=\; \frac{s(x,y) - s_{\min}}{\texttt{basinScale}},$$

where $s_{\min}$ is the deepest minimum centre (so the summit reads 0). One basin away $\Rightarrow b = 1 \Rightarrow v = 0.5$; the map saturates toward (never reaching) 1, so the far field always keeps a usable gradient. Crucially v never consults the surface's own value range, which is why adding minima no longer resizes basins. The mapping is monotone in distance, so it moves no optimum and changes nothing for the EA beyond the scale of the fitness numbers, and deterministic in `config`, so the main thread and the EA worker build identical surfaces. `isWin` tests `isWithinRadius(point, global, config.winRadius)`.

Notes. Tunables: `FAR_FALLOFF` (1.3; how steeply open ground climbs once outside a basin), `SMOOTH_K` (0.12; widest smooth-min blend, capped per-map against the tightest minima gap and reduced further if it would sink a local minimum below the global), `LOCAL_MIN_FLOOR_MIN/MAX` (0.18/0.55, as fractions of `basinScale`). Helper exports: `defaultLocalFloor(position)` (the seeded fractional floor an untouched node gets — shown in the create UI) and `effectiveFloor(minimum, basinScale)` (the floor in map units). Legacy maps without a `basinScale` fall back to `FALLBACK_BASIN_SCALE`.

functionProblem — engine/functionProblem.ts

Analytic benchmark surfaces (Sphere, Rastrigin, Eggholder, ...) behind the same `ProblemInstance` contract as hand-built maps.

```
interface FunctionSpec {          // fully defines a playable surface
  fn: string;                     // benchmark id, or 'procedural'
  cx: number; cy: number;         // where the optimum lands in [0,1]^2
  theta: number;                  // rotation (radians)
  sx: number; sy: number;         // anisotropic scale (zoom)
  rx: 1 | -1; ry: 1 | -1;         // reflections
  seed?: number;                  // procedural surfaces only
}
createFunctionProblem(spec, sharpenOverride?): ProblemInstance
randomFunctionSpec(rng?, { id?, category? }): FunctionSpec
randomSurfaceSpec(): FunctionSpec      // sentinel: re-randomise every load
resolveSpec(spec, rng?): FunctionSpec  // sentinel -> concrete procedural
proceduralFunction(seed): BenchmarkFn  // seeded fBm "Mystery Surface"
encodeFunctionCode(spec): string / decodeFunctionCode(code): FunctionSpec
```

`BENCHMARK_FUNCTIONS` holds ∼20 classic test functions (several BBOB-derived) grouped into `FUNCTION_CATEGORIES` (`simple | normal | complex | quirky`). `createFunctionProblem` grid-samples the affinely transformed surface once (64×64) to normalise values to $[0, 1]$ (85th-percentile ceiling, so corner extremes don't crush the colour range), applies a per-function sharpening exponent, and pins the win target to the lowest sampled point. Winning is *value*-based: `isWin` fires once `evaluate` $\leq$ `WIN_VALUE_EPS` (0.04), not at an exact coordinate; `metadata.winRadius` reports the measured extent of that zone for replay overlays. It also sets `displayExponent` $= 0.55$ so the heatmap compresses the highs and keeps more colour near the optimum (a display-only curve — see `ProblemInstance`); the sharpening exponent above shapes `evaluate` itself and is separate. Everything is deterministic in the spec (procedural surfaces in `seed`), so the main thread and the EA worker rebuild identical surfaces.

buildContours — engine/contours.ts

```
function buildContours(
  evaluate: (x: number, y: number) => number, // surface, values in [0,1]
  levels: number[],                           // iso-values to trace
  resolution: number = 80,                    // grid density
): ContourLevel[]
```

Marching-squares isoline extraction. Samples the surface once on a `resolution` × `resolution` grid over $[0, 1]^2$, then emits, per iso-level, the line segments where the surface crosses that level.

Returns. One `ContourLevel` (`level` + `segments`) per requested level; each `ContourSegment` is `x1, y1, x2, y2` in the same normalized $[0, 1]^2$ space.

geometry helpers — engine/geometry.ts

```
euclideanDistance(a: Coordinate, b: Coordinate): number
isWithinRadius(point: Coordinate, center: Coordinate, radius: number): boolean
closestMinimumDistance(point: Coordinate, minima: Coordinate[]): number
```

Plain 2-D geometry on normalized coordinates. `closestMinimumDistance` returns `Infinity` for an empty minima list (used by create mode's spacing check).

sampleGradient / sampleGradientRgb — engine/colorScale.ts

`sampleGradient(t: number): [r, g, b]` — shared colour ramp used by heatmap, contours and charts: a full-spectrum gradient from deep violet ($t = 0$, the summit) through blue, cyan, green, yellow and orange to deep red ($t = 1$, the valley), defined by an easily editable stop list. `sampleGradientRgb(t)` returns the same colour as a CSS `rgb()` string.

valueToHeight — engine/height.ts

`valueToHeight(value: number): number` — presentation-only inversion $1 - \texttt{value}$ (clamped to $[0, 1]$). Internally 0 = summit and 1 = valley; the player is shown "height" with 1 = summit. Use only for numbers displayed to the player — never feed the result into colour sampling or EA logic.

encodeMap / decodeMap / generateRandomMap — engine/mapCodec.ts

```
encodeMap(config: MapConfig): string  // URL-safe base64 share code
decodeMap(code: string): MapConfig    // throws Error('Invalid map code')
generateRandomMap(size: MapSizeId = 'medium'): MapConfig
```

Map share-codes; format specification in Chapter 7. `decodeMap` rejects unknown format versions and any unparsable input with a single `Invalid map code` error; missing `wr`/`t` fields fall back to 0.04 / `Date.now()`, and a missing `bs` to the Medium preset's `basinScale`. `generateRandomMap` takes a size preset (`MAP_SIZES`: `'small' | 'medium' | 'large' | 'huge'`, each fixing a minima count, win radius, min spacing and `basinScale`), places that many minima with the preset's spacing and promotes a random one to global.

problemCode — engine/problemCode.ts

```
type ProblemSource =              // plain data - survives postMessage
  | { kind: 'map'; config: MapConfig }
  | { kind: 'fn';  spec: FunctionSpec };
decodeProblem(code: string): DecodedProblem  // throws on invalid codes
decodeProblemOrNull(code?: string): DecodedProblem | null
buildProblemFromSource(source: ProblemSource): ProblemInstance
```

Uniform entry point for *any* PeakFinder code: function codes carry a `k:'fn'` marker and are tried first; anything else is treated as a map code. `DecodedProblem` bundles the rebuilt `problem`, a display `label` ("Map #ABC123" / "f Rastrigin"), the serializable `source` for the EA worker, and `winTarget`/`winRadius` for overlays. A "random every time" sentinel spec is resolved once here on the main thread, so the worker rebuilds the same concrete surface for the whole session.

codeClipboard — engine/codeClipboard.ts

`copyCode(text): Promise<void>` / `pasteCode(): Promise<string>` — clipboard helpers with an in-app fallback: every copy also lands in `localStorage` (`bs.lastCopiedCode`), and paste falls back to that value when the system clipboard is unreadable (Firefox web pages, insecure origins).

4.3 EA engine (`engine/ea/`)

`createEAStepper` (worked example) — `engine/ea/evolutionaryAlgorithm.ts`

```
function createEAStepper(
  problem: ProblemInstance,
  config:  EAConfig,
  seed?:   number,          // seeds the LCG rng - same seed, same run
): EAStepper

interface EAStepper {
  step: (n: number) => StepResult;  // n = 0 reports without advancing
  currentGenIndex: () => number;
}
type StepResult =
  | { type: 'generation'; generation: Generation; replay?: ReplayFrame[] }
  | { type: 'solved'; generation: Generation; totalGenerations: number;
      replay?: ReplayFrame[] }
  | { type: 'exhausted'; totalGenerations: number; best: Individual };
```

Creates a resumable EA. The generation loop: sort best-first → copy elite (top 5%, ≥ 1) → select parents → crossover (gated by `crossoverRate`) → mutate (gated by `mutationRate`) → clamp to bounds → evaluate → win check. The EA counts as solved once $\max(2, \lceil \texttt{populationSize} \cdot \texttt{winPopulationFraction} \rceil)$ individuals sit inside the win radius simultaneously.

Notes. The `replay` on each result holds the frames of the breeding that *produced* the reported generation, so the replay's final frame matches that generation's dots. `runEA(problem, config, callbacks, seed?)` is the non-interactive wrapper that steps to completion, reporting through `EACallbacks` (`onGeneration` / `onSolved` / `onExhausted`). The module-level `WIN_POPULATION_FRACTION` (0.10) is only a fallback for configs missing `winPopulationFraction`.

`SELECTION_STRATEGIES`, `CROSSOVER_STRATEGIES`, `MUTATION_STRATEGIES` — `engine/ea/operators.ts`

Strategy registries mapping each `*Strategy` string to its implementation:

	Strategy	Behaviour
selection	`tournament`	Draw $k = 3$ random candidates, return the fittest.
	`roulette`	Fitness-proportionate with inverted weights (`maxFit − fitness`), so lower fitness wins more often.
	`elitist`	Uniform pick from the top 20% only.
crossover	`uniform`	Each gene (x, y) drawn independently from either parent.
	`arithmetic`	Blend $\alpha A + (1-\alpha)B$ with random α.
	`singlePoint`	One parent contributes x, the other y.
mutation	`gaussian`	Normally-distributed offset (Box–Muller).
	`uniform`	Flat offset within ±strength.
	`cauchy`	Heavy-tailed offset — occasional long jumps help escape local minima.

All mutation offsets are scaled by the decayed strength $\texttt{mutationStrength} \cdot \texttt{mutationDecay}^{gen}$.

Notes. `CROSSOVER_STRATEGIES_RECORDING` is the single source of truth: those variants additionally return the random choices made (`CrossoverResult` — blend factor α / per-gene source), which the replay log needs and which cannot be reconstructed from the child position once mutation has been applied. The plain `CROSSOVER_STRATEGIES` are thin coordinate-only wrappers that draw identical RNG values.

individual helpers — engine/ea/individual.ts

```
createRandom(problem, rng): Individual  // uniform in problem.bounds
evaluate(position, problem): Individual // wraps fitness + isSolution
clamp(coord, problem): Coordinate       // clip to problem.bounds
```

eaReplayLog — engine/ea/eaReplayLog.ts

`buildReplayFrames(before, nextGen, eliteCount, config, solutionThreshold, breeding?)` builds the frame list for one generation; `snapshot(ind, id)` freezes an individual with a stable per-replay id. `ReplayFrame` is a discriminated union on `phase`; every frame carries `headline` and `description` (ready-to-display prose) plus the population as `IndividualSnapshot[]`:

Phase	Extra payload
`sorted`	population ranked best-first
`elite`	`eliteIds`; `nextGen` (the elites copied so far)
`selection`	`strategy`; tournament: `candidateIds`, `winnerId`; roulette: normalized `weights`; elitist: `eliteTopCount`
`breeding`	`parentAId`/`parentBId`, `child`, `crossoverResult` (pre-mutation position), `crossoverStrategy`, `alpha`/`geneSource`, `didCrossover`
`mutating`	`childId`, `didMutate`, `before`/`after`/`delta`
`newGen`	the completed next generation
`winCheck`	`solutionIds`, `count`, `threshold`, `solved`

Notes. The breeding/mutating frames show the *real* first non-elite child: a `BreedingRecord` (parents, gate outcomes, pre-mutation position) is captured live inside the stepper's breeding loop, because those random choices cannot be reconstructed afterwards. The union also declares an `initial` phase, but `buildReplayFrames` never emits it — the stepper hands over an already-sorted population.

4.4 Worker protocol (engine/ea/ea.worker.ts)

WorkerInMessage / WorkerOutMessage (worked example) — types/ea.ts

The EA runs in a dedicated Web Worker (engine/ea/ea.worker.ts); one stepper persists in worker scope between messages. The main thread sends WorkerInMessages and receives WorkerOutMessages:

Direction	Type	Payload and meaning
in	START	config: EAConfig, source: ProblemSource — rebuilds the problem and creates the stepper. Does *not* step, and posts nothing back (generation 0 arrives with the first STEP, avoiding a duplicate).
in	STEP	count: number — advance up to count generations.
in	STOP	Discard the stepper.
out	GENERATION	generation: Generation, replay?: ReplayFrame[] — the replay of the breeding that produced this generation rides along on the same message.
out	SOLVED	generation, totalGenerations, replay?.
out	EXHAUSTED	totalGenerations, best: Individual — maxGenerations reached without solving.
out	ERROR	message: string — e.g. the problem could not be rebuilt from source.

4.5 Hooks (hooks/)

useEARunner (worked example) — hooks/useEARunner.ts

```
const ea = useEARunner();
ea.init(source, eaConfig); // create worker, send START - does NOT run
ea.step(n);                // advance n generations (default 1)
ea.stop(); ea.reset();
// state: ea.status, ea.generations, ea.currentGeneration, ea.best,
//        ea.totalGenerations, ea.latestReplay, ea.errorMessage
```

React-side owner of the EA worker. source is a serializable ProblemSource; config defaults to DEFAULT_EA_CONFIG. EAStatus = 'idle' | 'running' | 'solved' | 'exhausted' | 'error'. latestReplay holds the ReplayFrame[] of the most recent step batch, consumed by the replay overlay.

Notes. Once solved, totalGenerations is locked to the solving generation — further probes keep the worker stepping, but no longer advance the "solved in X generations" tally. start is currently an alias of init. Worker errors terminate the worker and surface in errorMessage.

replay playback — src/hooks/useReplayPlayer.ts

There is no PeakFinder-specific replay hook: EAReplayOverlay feeds its ReplayFrame[] to the shared, frame-agnostic useReplayPlayer<Frame>(frames, autoplayIntervalMs?), which provides currentFrame, frameIndex, totalFrames, isFirst/isLast, next/prev/goTo and play/pause autoplay.

useGameMap — hooks/useGameMap.ts

Create-mode map state.

```
const map = useGameMap(size: MapSizeId = DEFAULT_MAP_SIZE); // 'medium'
// state:   minima, mapId, maxMinima, minSpacing, winRadius, isFull,
//          hasGlobal, globalMinimum
// actions: addMinimum(pos), removeMinimum(id), setGlobalMinimum(id),
//          setFloor(id, floor | undefined), clearAll(),
//          getMapConfig(), getCode()
```

Create mode uses the very same size presets a random map does (`MAP_SIZES`, `engine/mapCodec.ts`): `size` fixes how many minima fit (`maxMinima` = the preset's upper bound), how close together they may stand (`minSpacing`), how tight the summit is (`winRadius`) and the `basinScale` baked into the config. `addMinimum` silently rejects placements beyond `maxMinima` or closer than `minSpacing` to an existing minimum. `setFloor(id, undefined)` reverts a node to its random default depth. `clearAll` also mints a fresh `mapId`, so each created map saves as its own entry. `getCode()` = `encodeMap(getMapConfig())`.

Notes. Changing `size` mid-build never discards what is already placed — it only constrains what may be added from here on, so shrinking below the current count simply reports the map as `isFull`.

usePlaySession — hooks/usePlaySession.ts

```
const { probes, status, bestProbe, probe, reset } =
  usePlaySession(problem: ProblemInstance | null);
```

Play-mode session state. `probe(position)` evaluates the problem and appends a `ProbeResult`; `status` is `'idle' | 'playing' | 'won'` and stays `'won'` once won (probing remains possible for post-win exploration). `bestProbe` is the probe with the lowest value so far.

useSavedMaps — hooks/useSavedMaps.ts

```
const { savedMaps, saveMap, removeMap, renameMap } = useSavedMaps();
interface SavedMap { id; name?; code; createdAt; minimaCount }
```

The player's saved-map library — a thin domain wrapper around the shared `useSavedLibrary` hook, persisted in `localStorage` under `bs.savedMaps`. `saveMap(config: MapConfig)` stores the encoded share code; saving an id that already exists is a no-op.

4.6 Key components (components/game/)

GameMap (worked example) — components/game/shared/GameMap.tsx

The central square map canvas (grid + heatmap + minima dots + overlays).

```
minima?: Minimum[]; showMinima?: boolean; highlightGlobal?: boolean;
evaluateFn?: (x, y) => number;  // enables the HeatmapLayer
heatmapConfig?: Partial<HeatmapConfig>;
revealPoints?: Coordinate[];    // heatmap only shown near these
exclusionRadius?: number;       // dashed rings around minima (create mode)
onMapClick?: (coord: Coordinate) => void;  // normalized coords
onMinimumClick?: (id: string) => void; selectedId?: string | null;
overlayLabel?: string; className?: string; children?: ReactNode;
```

Clicks are converted to normalized $[0,1]^2$ coordinates; clicks on a minimum dot go to `onMinimumClick` instead of `onMapClick`. Probe markers, win rings etc. are passed as absolutely-positioned `children`.

ContourLayer / HeatmapLayer — components/game/shared/

Both render a surface visualization from `{ evaluate, config?, revealPoints? }`. Reveal semantics: `undefined` reveals the whole map; an array (even an empty one) reveals only circles of `revealRadius` around its points. `GameMap` currently renders the heatmap; `ContourLayer` (SVG marching-squares isolines, levels packed densely near the summit) remains available for contour-style views. Defaults: `DEFAULT_CONTOUR_CONFIG = { lineCount: 24, spacingExponent: 2.5, resolution: 100, revealRadius: 0.1 }`; `DEFAULT_HEATMAP_CONFIG = { resolution: 1080, opacity: 0.82, revealRadius: 0.05, valueExponent: 1 }`. `valueExponent` is the display curve applied as `value` $^{\texttt{valueExponent}}$ before colouring; 1 sends the value straight to the ramp (what surface maps use), while callers pass a problem's `displayExponent` through it (benchmark functions use 0.55). It only affects colour — never readings, wins or the EA.

EAReplayOverlay / ReplayMap / IndividualList — components/game/vs-ea/

Full-screen replay modal: `EAReplayOverlay({ frames, onClose })` drives the shared `useReplayPlayer` through the frame list. `ReplayMap` renders the population dots plus per-phase decorations: `highlightIds`/`dimIds`/`markerIds`/`solutionIds` sets, `customOpacities` (roulette weights), `extraDots` (the bred child), `arrow` (mutation offset), `parentLine` (arithmetic crossover) and `geneLines` (uniform/single-point gene sources). Dots glide with `DOT_MOVE_DURATION` = 1000 ms. `IndividualList` lists the population with role badges (`ELITE`, `PARENT A/B`, `CHILD`, `WIN`, `WINNER`, `CANDIDATE`, `ELIGIBLE`) via `roles: Map<id, IndividualRole>` and optional per-row `annotations`.

FitnessChart, ModeSelector, CodeModal — components/game/shared/

`FitnessChart({ series, yMax?, compact?, yLabel?, hoveredIndex?, onHover?, markers? })` — SVG line chart of probe/EA values over time (y-axis shown as the player-facing "height"). `ModeSelector({ onSelect })` — the Create / Play / Vs EA landing picker. `CodeModal({ code, mapId, onClose, onPlayNow? })` — share-code dialog with clipboard copy via `copyCode`.

SavedMapsSidebar / SavedFunctionsSidebar — components/game/shared/

Two self-contained left-edge drawers (no props). `SavedMapsSidebar` lists the player's saved maps (`useSavedMaps`) with rename/delete; clicking an entry copies its share code. `SavedFunctionsSidebar` lists the analytic benchmark functions grouped by category; copying a function mints a *fresh* random transform each time, and "Random every time" copies the sentinel spec that re-randomises the whole surface on every load.

VsEAMode and overlays — components/game/vs-ea/

`VsEAMode` wires the race: it owns the `EAConfig` (initialised to the `'normal'` preset), steps the EA per player probe, and mounts `EASettingsPanel` (difficulty presets from `EA_PRESETS` plus advanced sliders), `EAWinOverlay` / `SecondSolveOverlay` (EA won / player beat its solution on the retry) and `GenerationReplayOverlay` (dot movement across whole generations, as opposed to the phase-by-phase `EAReplayOverlay`).

Chapter 5

Maze Explorer Module (mazeGame)

All paths relative to `modules/mazeGame/`. The module mirrors the BattleShips structure with a *discrete* genome: an individual is a fixed-length move sequence instead of a point.

5.1 Type contracts (`types/`)

Maze core types (worked example) — types/maze.ts

```
interface Cell { x: number; y: number }  // grid coordinates
type Move = 0 | 1 | 2 | 3;          // indexes into MOVE_ARROWS / MOVE_DELTAS
type Path = Move[];                  // the genome: fixed-length move sequence
const MOVE_ARROWS = ['↑','→','↓','←'];
type WallRule  = 'waste' | 'break' | 'repair';
type FitnessFnId = 'manhattan' | 'geodesic' | 'length' | 'novelty';

interface Grid { cols: number; rows: number; walls: Uint8Array }
interface SerializedMaze extends Grid { start: Cell; goal: Cell }
interface WalkResult {           // the phenotype of a Path
  visited: Set<number>;          // visited cell indices (y*cols + x)
  trail: Cell[];                 // cell per step; length = path.length + 1
  finalCell: number;             // where the agent settled
  reachedGoalAt: number;         // step it first stood on the goal, or -1
  crashedAt: number;             // wall-crash step ('break' rule only), or -1
}
interface MazeProblem {
  grid: Grid; cols; rows; start: Cell; goal: Cell;
  geodesicField: Int32Array;     // BFS distance to goal; -1 = unreachable
  pathLength: number;            // auto-sized genome length L
  fitnessFnId: FitnessFnId; wallRule: WallRule;
  evaluate: (walk: WalkResult) => number;  // lower = better, 0 = optimal
  isWin: (walk: WalkResult) => boolean;
  metadata: { seed: number; shortestPath: number };
}
```

Notes. `Grid.walls[i]` holds an *open-passage* bitmask per cell $i = y \cdot \texttt{cols} + x$ (N=1, E=2, S=4, W=8; a set bit means the passage is open — `MOVE_WALL_BIT`); `cellIndex(x, y, cols)` maps 2-D coordinates to flat indices. `SerializedMaze` decouples a hand-built creator maze from procedural generation (start/goal are arbitrary). `MazeProblem` parallels BattleShips' `ProblemInstance`, but `evaluate`/`isWin` take a simulated `WalkResult` instead of an (x, y) coordinate. `WallRule` is the constraint-handling strategy for blocked moves: `waste` loses the move, `break` crashes and freezes the agent (death penalty), `repair` rewrites the gene to a random open direction and takes it (Lamarckian).

EAConfig / DEFAULT_MAZE_EA_CONFIG types/ea.ts

Discrete-EA configuration. Shape mirrors the BattleShips `EAConfig`, plus three maze-only fields (`fitnessFnId`, `wallRule`, `pathLengthFactor`); an `Individual` carries a `path` and its cached `walk` instead of a position.

Field	Default	Meaning
`populationSize`	100	Individuals per generation
`maxGenerations`	300	Hard stop
`crossoverRate`	0.85	Probability of recombination
`mutationRate`	0.95	Probability of mutating a child
`mutationStrength`	0.06	Per-gene re-pick *probability* (not a perturbation magnitude)
`mutationDecay`	1.0	Per-generation strength multiplier (no decay — keeps exploration alive)
`winPopulationFraction`	0.08	Fraction that must reach the goal ⇒ solved
`selectionStrategy`	`'tournament'`	`| 'roulette' | 'elitist'`
`crossoverStrategy`	`'singlePoint'`	`| 'uniform'`
`mutationStrategy`	`'point'`	`| 'segment'`
`fitnessFnId`	`'geodesic'`	Which fitness function the problem evaluates against
`wallRule`	`'waste'`	Blocked-move handling (`| 'break' | 'repair'`)
`pathLengthFactor`	3.5	Genome length as a multiple of the shortest path ($\geq$ 1; baked in at build time — changing it restarts the run)

5.2 Engine (`engine/`)

mazeGen engine/mazeGen.ts

```
generateMaze(cols, rows, seed, { braid?, openness? }): Grid
pickRandomStartGoal(grid, rng, minFrac = 0.6): { start; goal }
gridFromEdgeWalls(cols, rows, isHWall, isVWall): Grid
```

Seeded procedural generation — identical seed ⇒ identical maze (the basis of the fitness-comparison experiment). `generateMaze` runs an iterative recursive backtracker (every cell reachable), then two seeded post-passes that only ever *remove* walls, so connectivity is preserved: `braid` opens a fraction of dead-ends into loops, `openness` removes a fraction of the remaining interior walls (punching side doors into the backtracker's long corridors, creating parallel routes the population can split across).

Notes. `pickRandomStartGoal` places the goal anywhere and draws the start from cells whose corridor distance to the goal is at least `minFrac` of the maze's BFS diameter — never trivially close. `gridFromEdgeWalls` converts the maze creator's edge-wall model into a `Grid`. `mazeToAscii` is a debug-only ASCII renderer.

geodesic — engine/geodesic.ts

```
computeGeodesic(grid: Grid, goal: Cell, start: Cell): GeodesicResult
interface GeodesicResult {
  field: Int32Array;         // corridor distance to goal; -1 = unreachable
  diameter: number;          // largest finite distance in the field
  shortestFromStart: number; // start -> goal distance
}
```

BFS distance field flooded outward from the goal through open corridors, computed once at problem build time. Backs the `geodesic` and `length` fitness functions, genome auto-sizing, and `pickRandomStartGoal`.

mazeProblem — engine/mazeProblem.ts

`createMazeProblem(opts: BuildMazeOptions): MazeProblem` — generates the maze (or accepts an explicit `grid` for creator mazes), floods the geodesic field, auto-sizes the genome length $L = \min(\texttt{MAX\_PATH\_LENGTH}, \lceil \text{shortest} \cdot \texttt{pathLengthFactor} \rceil)$, and wires up the chosen fitness function. Changing only `fitnessFnId` for the same seed yields the *same* maze with a different `evaluate` — the comparison demo. All fitness functions are "lower = better", normalized to roughly $[0, 1]$, and score the agent's *final* cell (the goal absorbs), so a lucky fly-through earns no credit:

Id	Scoring
`manhattan`	Straight-line distance of the final cell to the goal — ignores walls, so it is *deceptive*: the population piles up in the dead-end nearest the goal as the crow flies, and stalls.
`geodesic`	Corridor (BFS) distance of the final cell to the goal, normalized by the maze diameter — smooth and solvable.
`length`	Geodesic plus a reward for reaching the goal early: non-solvers score in $[0.5, 1]$, solvers in $[0, 0.5)$ scaled by `reachedGoalAt` — reaching dominates, shorter wins among solvers.
`novelty`	Placeholder (constant 1): novelty depends on the whole population + archive, so the stepper applies a `NoveltyScorer` that overwrites each individual's fitness.

Notes. `isWin` is `reachedGoalAt >= 0` regardless of fitness function. Defaults: `DEFAULT_BRAID` = 0.5, `DEFAULT_OPENNESS` = 0.25 (too high and Manhattan stops being deceptive), `MAX_PATH_LENGTH` = 600 (exported so the creator can warn when a maze's shortest path exceeds what a genome can hold).

`createNoveltyScorer (worked example)` — `engine/ea/novelty.ts`

```
function createNoveltyScorer(
  cols: number, rows: number, k: number = DEFAULT_K,
): NoveltyScorer
```

Novelty-search infrastructure — the canonical maze-deception demo (Lehman & Stanley): objective-chasing gets trapped, pure exploration floods the maze and stumbles onto the exit. Instead of rewarding closeness to the goal, each individual is rewarded for ending up somewhere *few* others (now or in the past) have ended up. Used when `FitnessFnId = 'novelty'`; because it depends on the whole population plus an archive, the stepper calls `scorer.score(pop)` each generation instead of a per-individual `evaluate`.

Notes. Behaviour descriptor = the final cell the agent settles on. `score` sets each individual's fitness to $1 - \min(1, \text{novelty}/\text{maxDist})$, where novelty is the mean Euclidean distance to the k nearest behaviours ($k = 15$ by default, own behaviour excluded) among the current population plus the archive. Archive admission: every *newly explored* distinct final cell is added (deduplicated by cell index); `archiveSize()` exposes the count for visualization.

`individual helpers / walkPath` — `engine/ea/individual.ts`

```
walkPath(problem, path, rng?): WalkResult   // simulate the phenotype
evaluate(path, problem, rng?): Individual   // walk + score + isSolution
createRandom(problem, rng): Individual      // random genome of length L
repair(path, problem, rng): Path            // enforce genome length L
randomMove(rng): Move
```

`walkPath` executes the genome move by move. The trail is always `path.length + 1` cells (uniform for animation): a blocked move keeps the agent in place under `waste`, freezes it for the rest of the walk under `break` (recorded in `crashedAt`), or rewrites the gene to a random open direction under `repair` (Lamarckian — `evaluate` therefore copies the genome first, so shared parent arrays are never mutated). The goal *absorbs*: once reached, the agent stays there, which is what makes the final cell a meaningful fitness target. `repair` is the length-invariant safety net (truncate / pad with random moves).

`SELECTION_STRATEGIES, CROSSOVER_STRATEGIES_RECORDING, MUTATION_STRATEGIES` — `engine/ea/operators.ts`

Selection is ported verbatim from BattleShips (tournament $k = 3$ / roulette with inverted weights / elitist top 20% — it only reads `.fitness`). Crossover and mutation work on the move sequence:

	Strategy	Behaviour
crossover	`singlePoint`	Follow parent A up to a random cut ($1..L-1$), then switch to parent B; records `cut`.
	`uniform`	Each gene independently drawn from either parent; records the per-gene `geneMask`.
mutation	`point`	Each gene independently re-picked with probability $\texttt{mutationStrength} \cdot \texttt{mutationDecay}^{gen}$.
	`segment`	Re-randomize one contiguous block whose max length scales with the decayed strength (a "kink" the path forks at).

Notes. Both operator families are recording variants (`CrossoverResult` / `MutationResult` with `mutatedIndices`) — the replay log needs the splice cut / gene mask / mutated positions, which cannot be reverse-engineered from the child genome.

createMazeEAStepper — engine/ea/evolutionaryAlgorithm.ts

`createMazeEAStepper(problem, config, seed?): EAStepper` — same resumable-stepper contract and `StepResult` union as BattleShips (Chapter 4), with two maze-specific additions:

- `updateConfig(nextConfig, nextProblem?)` — swap in new tuning *without* discarding the population. When `nextProblem` is given (fitness function or wall rule changed), every genome is kept but re-walked and re-scored under the new problem.
- Novelty runs use a solution threshold of 1 (a single explorer finding the exit counts as solved) — novelty spreads the population out, so it never piles a fraction onto the goal. All other fitness functions use $\max(2, \lceil \texttt{populationSize} \cdot \texttt{winPopulationFraction} \rceil)$.

Elitism is top 5% (≥ 1), as in BattleShips.

eaReplayLog — engine/ea/eaReplayLog.ts

Same phase set and frame structure as the BattleShips replay log (`sorted` → `elite` → `selection` → `breeding` → `mutating` → `newGen` → `winCheck`), re-based on paths: `IndividualSnapshot` carries `path`, `trail` and `finalCell` instead of a position (the map draws trails as spaghetti, the list renders genomes as arrow strings). Payload differences: `breeding` records `cut`/`geneMask` instead of α/`geneSource`; `mutating` records `beforePath`/`afterPath`/`mutatedIndices` instead of a coordinate delta.

mazeCodec — engine/mazeCodec.ts

```
mazeId(maze: SerializedMaze): string   // content-derived 6-char id
encodeMaze(maze: SerializedMaze): string
decodeMaze(code: string): SerializedMaze // throws Error('Invalid maze code')
```

Maze share-codes; format in Chapter 7. `mazeId` hashes the maze content (djb2 → base36), so identical mazes get identical ids — what the saved-maze library de-duplicates on. `decodeMaze` validates version, dimensions (2..64 per axis), wall string length and marker bounds, and throws a single `Invalid maze code` error otherwise.

5.3 Worker protocol (`engine/ea/maze.worker.ts`)

`WorkerInMessage / WorkerOutMessage` — `types/ea.ts`

Same shape as the BattleShips protocol; the out-messages (`GENERATION` / `SOLVED` / `EXHAUSTED` / `ERROR`) are identical apart from the maze-typed payloads. Differences on the in-side:

Direction	Type	Payload and meaning
in	`START`	`config: EAConfig`, `seed`, `cols`, `rows`, `maze?: SerializedMaze` — a hand-built creator maze takes precedence; otherwise the maze is generated procedurally from `seed`. The EA RNG is seeded with the same value, so on the same seed the *only* thing that varies between fitness functions is `evaluate` — the comparison demo.
in	`STEP`	`count: number`.
in	`UPDATE_CONFIG`	`config: EAConfig` — retune the running EA without restarting; takes effect on the next generation. Only a changed `fitnessFnId` or `wallRule` rebuilds the problem (and re-scores the population); everything else is pure tuning.
in	`STOP`	Discard the stepper.

Notes. `pathLengthFactor` is fixed for the worker's lifetime: the main thread sends a fresh `START` when it changes, so mixed-length populations can never occur.

5.4 Hooks and components

`useMazeEARunner` — `hooks/useMazeEARunner.ts`

```
const ea = useMazeEARunner();
ea.init({ seed, cols, rows, maze?, pathLengthFactor? }, config?);
ea.step(n); ea.updateConfig(config); ea.stop(); ea.reset();
// state: status, generations, currentGeneration, best,
//        totalGenerations, latestReplay, errorMessage
```

Mirrors BattleShips' `useEARunner` (same `EAState`, same solved-lock on `totalGenerations`, `start` again an alias of `init`); the differences are the `START` payload (`MazeRunParams` instead of a `ProblemSource`), the default config (`DEFAULT_MAZE_EA_CONFIG`) and the extra `updateConfig` for live retuning. Replay playback again uses the shared `useReplayPlayer` — there is no maze-specific replay hook.

`useSavedMazes` — `hooks/useSavedMazes.ts`

```
const { savedMazes, saveMaze, removeMaze, renameMaze } = useSavedMazes();
interface SavedMaze { id; name?; code; createdAt; cols; rows }
```

Saved-maze library over the shared `useSavedLibrary`, persisted in `localStorage` under `maze.savedMazes`. Entries are keyed by the content-derived `mazeId`, so saving an identical maze is a no-op; `saveMaze` returns the id either way.

MazeCanvas — components/shared/MazeCanvas.tsx

The shared maze renderer (walls, start/goal markers, overlays). Key props: `cols`/`rows`/`walls`/`start`/`goal` plus `trails` (population / player paths, drawn in order), `solidCells` (creator's painted blocks), `markers` (highlight dots, e.g. splice / mutated cells), `agents` (animated dots drawn above walls), `previewWall` (creator hover ghost), `showGrid`, and a fog-of-war cell set used by Solve mode to reveal the maze as it is explored.

MazeSetupScreen / MazeExperimentMode / MazeEASettingsPanel — components/experiment/

`MazeSetupScreen` — experiment entry: pick a fresh random maze (size / openness of your choosing) or a saved creation, then start. `MazeExperimentMode` — the run itself: owns the `EAConfig`, drives `useMazeEARunner`, and pins settings changes to the generation where they took effect as markers on the fitness chart. `MazeEASettingsPanel({ config, onConfigChange, shortestPath?, runStarted?, onClose })` — the `EAConfig` editors (also exported chrome-free as `MazeEASettingsControls`); the genome-length slider is locked once a run has started, since length is baked into every genome.

MazeEAReplayOverlay / MazePathMap / MazeIndividualList — components/experiment/replay/

Full-screen dissection of one generation — a re-skin of BattleShips' replay overlay with an identical phase set. `MazeEAReplayOverlay({ frames, problem, onClose })` steps the shared `useReplayPlayer`. `MazePathMap` draws every individual's walked trail over the maze (highlight/dim/marker/solution id-sets, `customOpacities` for roulette, `selectedId` to lift one path above the rest, splice-point cell markers). `MazeIndividualList` renders genomes as arrow strings with role badges, per-gene highlights (crossover source / mutated genes), `maxGenes` truncation and click-to-select (`onSelect`).

MazeCreateMode / MazePlayMode — components/create/, components/play/

`MazeCreateMode` — the maze editor: draw/erase walls on cell edges (elliptical edge hitboxes so free-form strokes don't grab stray orthogonal walls), place start/goal, size 4–32 per axis, export via `encodeMaze`. `MazePlayMode` — the player writes a move string with a D-pad and watches it walk (one step per 140 ms tick), edits it, and re-submits — experiencing genome, phenotype and mutation by hand; a ghost replays the previous attempt for comparison.

Chapter 6

Shooter Module (shooterGame)

All paths relative to `modules/shooterGame/`.

6.1 Types and constants (`shooter.types.ts`)

The single source of truth for every shooter type and tunable.

`DNA (worked example)` — `shooter.types.ts`

An agent genome is `type DNA = number[]` of length `DNA_LENGTH` (currently **8**); all genes lie in $[0, 1]$. Indices come from `DNA_INDEX` — never a magic number:

#	Gene	Effect in `updateAgent` (`game/core/gameLoop.ts`)
0	`AGGRESSION`	Weight of the chase vector — how eagerly the agent closes distance. Zeroed once inside `PREFERRED_RANGE`.
1	`DODGE_WEIGHT`	Per-frame *probability* of sidestepping a near bullet; the dodge's speed comes from `MOVEMENT_SPEED`.
2	`SHOOT_ACCURACY`	Aim scatter: $(\texttt{rand} - 0.5)(1 - \texttt{gene}) \cdot 1.2$ radians, so 1 = perfect.
3	`PREFERRED_RANGE`	Combat distance = $\texttt{gene} \cdot 300 + 100$ px.
4	`MOVEMENT_SPEED`	Speed = `AGENT_SPEED_BASE`+`gene`·`AGENT_SPEED_BONUS` = 40+`gene`·80 px/s.
5	`PREDICT_LEAD`	How far the agent leads the player's velocity when aiming.
6	`FIRE_RATE`	Cooldown interpolates `SHOOT_COOLDOWN_MIN`…`MAX` (higher gene = faster).
7	`BULLET_SPEED`	Bullet speed = `BULLET_SPEED_MIN` + `gene` · (`MAX` − `MIN`) = 150 … 900 px/s.

Notes. `DNA_LENGTH` is derived (`Object.keys(DNA_INDEX).length`), so adding a gene to `DNA_INDEX` is the only edit needed — but its trailing `// = 7` comment is stale and the real value is 8. `DNA_GENE_INFO` carries the player-facing label and tooltip of each gene: note that `AGGRESSION` is *labelled* "Pursuit" in Solo/Raidboss, because the gene only affects distance-closing and is switched off once in range; Horde's own aggression mechanic genuinely earns the word and keeps the "Aggression" label through its own descriptor override (`HORDE_BAR_GENES`). Do not "fix" either back. `DNA_NAMES` lists the keys in index order. Presets: `STARTER_DNA` (deliberately feeble — all 0.1 except `PREFERRED_RANGE` 0.3 and `BULLET_SPEED` 0.5) and `TUTORIAL_DNA` (an almost inert target: no special-casing in the game loop, the DNA alone makes it harmless).

GAME_CONFIG, ARENA shooter.types.ts

ARENA is the logical canvas (800 × 800 px, PADDING 20). GAME_CONFIG holds every gameplay constant:

Field	Default	Meaning
ROUND_DURATION	20	Seconds per round
POPULATION_SIZE	40	Individuals in the GA population
ELITE_COUNT	4	Top-N carried over unchanged
MUTATION_RATE	0.1	Per-gene mutation probability
MUTATION_STRENGTH	0.2	Max. per-gene perturbation
BULLET_SPEED	500	px/s — the player's default
BULLET_SPEED_MIN/MAX	150 / 900	px/s — range of the agent's DNA gene
BULLET_LIFETIME	2	Seconds before a bullet expires
SHOOT_COOLDOWN	0.4	Seconds between player shots
SHOOT_COOLDOWN_MIN/MAX	0.3 / 1.5	Range the agent's FIRE_RATE maps into
AGENT_SPEED_BASE	40	px/s the agent always has
AGENT_SPEED_BONUS	80	px/s scaled by MOVEMENT_SPEED
PLAYER_SPEED	320	px/s
PLAYER_RADIUS, AGENT_RADIUS	18	Collision radii
BULLET_RADIUS	5	Collision radius
NEAR_MISS_RADIUS	40	px — closer than this counts as "dodged"

Entities, state and stats — shooter.types.ts

```
interface Entity { position: Vector2D; velocity: Vector2D;
                   rotation: number; radius: number; health: number }
interface PlayerState extends Entity { id: 'player' }
interface AgentState  extends Entity { id: 'agent'; dna: DNA;
                                       stats: RoundStats; pendingBullet?: Bullet }
interface Bullet { id: string; position: Vector2D; velocity: Vector2D;
                   ownerId: 'player' | 'agent'; lifetime: number; radius: number;
                   homing?: boolean;   // Homing-Rounds mod
                   bounces?: number }  // Ricochet mod: wall bounces left
interface RoundStats { hitsLanded; hitsReceived; bulletsFired; timeAlive;
                       dodgedBullets; distanceSum; distanceSamples }
type GamePhase = 'idle' | 'playing' | 'roundEnd' | 'evolving';
interface GameState {
  phase: GamePhase; roundTimer: number; roundNumber: number;
  player: PlayerState; agent: AgentState; bullets: Bullet[];
  population: Population | null;
  lastPlayerFrame: PlayerGhostFrame | null;
  lastAgentFrame:  AgentGhostFrame  | null;
  crossoverExample: CrossoverExample | null;
}
interface Individual { dna: DNA; fitness: number }
interface Population { generation: number; individuals: Individual[];
                       bestFitness: number; avgFitness: number }
```

`emptyStats()` mints a zeroed `RoundStats`. `InputState` is the keyboard/mouse/touch snapshot (`up`, `down`, `left`, `right`, `shoot`, `mouseX`, `mouseY`). `PlayerStats` is the player's own tuning (`bulletSpeed`, `moveSpeed`, `shootCooldown`) — the surface the run mods modify. `CrossoverExample` (`parentA`, `parentB`, `geneOrigins`, `type`) records one recombination for the DNA-reveal UI.

Notes. `GameState` holds only the *last* ghost frame of each side (`lastPlayerFrame` / `lastAgentFrame`); the full `PlayerGhostFrame[]` recording is accumulated by `ShooterCanvas`, not carried in the state. `Individual` is just DNA + fitness — round stats live on `AgentState.stats` during play.

Player ghosting — shooter.types.ts

```
interface PlayerGhostFrame { position: Vector2D; velocity: Vector2D;
                             rotation: number; shot: boolean; time: number }
interface AgentGhostFrame  { position: Vector2D; rotation: number; shot: boolean }
interface PlayerGhost {
  frames: PlayerGhostFrame[];
  roundDuration: number;
  bulletSpeed?:  number;    // player's real bullet speed when recorded
  activeModIds?: string[];  // behaviour mods shape the ghost's shot plan too
}
```

A `PlayerGhost` is a recording of one played round — the training data the Solo GA evolves against. `bulletSpeed` and `activeModIds` are captured so the replayed player fights exactly as hard as the real one did (falling back to `GAME_CONFIG.BULLET_SPEED` when absent).

Tutorial persistence — `shooter.types.ts`

`TUTORIAL_COMPLETED_KEY` (`shooter.tutorial.completed`), `HORDE_TUTORIAL_COMPLETED_KEY` (`horde.tutorial.completed`) and `RAIDBOSS_TUTORIAL_COMPLETED_KEY` (`raidboss.tutorial.completed`) are the `localStorage` flags each mode sets after a full tutorial run. `hasCompletedAnyTutorial()` returns true if *any* of them is set: the controls are identical in all three modes, so a player who finished one tutorial gets the `TutorialChooserModal` (practice round vs. explainer) everywhere instead of a forced repeat.

6.2 Core loop (`game/core/`)

update (worked example) — `game/core/gameLoop.ts`

```
function update(
  state:        GameState,
  dt:           number,
  input:        InputState,
  playerStats:  PlayerStats,
  activeModIds: string[] = [],
): GameState
```

The **pure** per-frame simulation step — no React, no globals beyond the module's own scratch state. Returns the next `GameState`, and is a no-op (returns `state` unchanged) unless `phase === 'playing'`. Order per frame: tick the round timer (hitting 0 ⇒ `phase: 'roundEnd'`) → move the player from `input` → run `updateAgent` (the DNA → behaviour mapping of the table above) → fire queued/triggered shots via `computeShotPlan(activeModIds)` → advance bullets (homing, wall bounces, shield blocks) → resolve collisions and update `RoundStats`.

Side effects. Module-level scratch state (bullet id counter, both shoot cooldowns, the dodged-bullet set, queued burst shots, the orbit-shield angle) lives outside `GameState` because it must not influence the GA. `resetGameLoop()` clears all of it between rounds/runs; `getShieldAngle()` exposes the shield rotation for the renderer.

Notes. `updateAgent` reuses pre-allocated scratch vectors (`_agToP`, `_agChase`, ...) so the hot path allocates nothing.

renderer, vec, entities — `game/core/`

`renderer.ts` holds the canvas draw calls (`drawArena` — cached to an offscreen canvas — `drawPlayer`, `drawAgent`, `drawBullets`, ...). `vec.ts` is the 2-D vector toolbox (`add`, `sub`, `scale`, `length`, `normalize`, `angle`, `clamp`, `perpendicular`, `zero`, ...) plus the `Vector2D` type. `game/entities/` (`player.ts`, `agent.ts`, `bullet.ts`) holds thin construction helpers, and `game/makeGameState.ts` exports `makeInitialGameState(settings)` — a fresh `GameState` from `ShooterSettings`.

6.3 Genetic algorithm (`game/ga/`)

Unlike the map and maze EAs, this GA **maximizes** fitness.

evolve (worked example) — game/ga/evolution.ts

```
function evolve(
  population:         Population,
  agentFitness?:      number,   // real round fitness of the agent just fought
  mutationRate:       number = GAME_CONFIG.MUTATION_RATE,
  mutationStrength:   number = GAME_CONFIG.MUTATION_STRENGTH,
  crossoverType:      'uniform' | 'single-point' = 'uniform',
  injectionDeviation: number = 0.3,
): Population
```

Produces the next generation. When `agentFitness` is given it overwrites individual 0's fitness first — that is how the *real* round result (rather than a simulated one) enters the GA. Then: sort best-first → copy the top `GAME_CONFIG.ELITE_COUNT` (4) unchanged → fill the middle with offspring (tournament select $k = 3$ → crossover → per-gene mutate) → append `INJECTION_COUNT` (4) fresh variants of the *best* DNA, each gene jittered by $\pm$`injectionDeviation`. Population size is preserved: `elites` + `offspring` + `injected`.

Notes. Crossover defaults to `uniform` (per-gene coin flip); `single-point` splices at a random cut in $1 \ldots$ `DNA_LENGTH` $- 1$. Mutation offsets are uniform in $\pm$`mutationStrength` and clamped to $[0, 1]$. Selection/mutation use `Math.random` directly — only the ghost simulation is seeded (see `createGhostSim`). The diversity injection is what keeps the population from collapsing onto one elite after many rounds. `getNextAgent(population): number[]` returns `individuals[0].dna` — the DNA the player faces next, i.e. the best individual, since elites are placed first.

fitness — game/ga/fitness.ts

```
calculateFitness(stats: RoundStats): number
calculateRaidbossFitness(stats: RoundStats, roundDuration: number): number
normalizeFitness(fitnesses: number[]): number[]
```

Higher = better. The Solo score is the hit balance plus a flat outcome bonus:

$$\texttt{fitness} = 100\,(\texttt{hitsLanded} - \texttt{hitsReceived}) + \begin{cases} +120 & \text{net} > 0 \\ -80 & \text{net} < 0 \\ 0 & \text{otherwise} \end{cases}$$

`calculateRaidbossFitness` adds a survival term $(\texttt{timeAlive}/\texttt{roundDuration}) \cdot 60$ — deliberately capped at $+60$ so the hit balance stays the dominant signal, while a boss that held out under pressure still scores above one that died early with the same balance. `normalizeFitness` min–max scales a list into $[0, 1]$, returning a uniform $1/n$ when every value is equal.

population — game/ga/population.ts

```
randomDNA(): number[]                          // DNA_LENGTH genes, uniform [0,1)
initPopulation(starterDna = STARTER_DNA,
               size = GAME_CONFIG.POPULATION_SIZE): Population
updatePopulationStats(population): Population   // recompute best/avg fitness
getBestIndividual(population): Individual
```

`createGhostSim (worked example)` — `game/ga/evolution.ts`

```
function createGhostSim(dna: DNA, ghost: PlayerGhost): GhostSim
interface GhostSim {
  readonly agent:         SimAgent;
  readonly agentBullets:  readonly SimBullet[];
  readonly playerBullets: readonly SimBullet[];
  readonly stats:         RoundStats;
  readonly frame:         number;   // index of the next ghost frame
  readonly done:          boolean;
  step(): void;                     // advance one ghost frame (1/60 s)
}
```

A stepwise simulation of one candidate DNA against a recorded round — and the single source of the sim rules: the fitness evaluation runs it to completion, while `EvolutionReplayOverlay` steps it frame by frame to draw it. Sharing one implementation is what makes the replay show the *same* fight the selection actually scored.

Notes. The RNG is a seeded LCG whose seed is derived from the DNA itself (`dnaSeed`, an FNV-style fold), so the same DNA always produces the same fight — evaluation and replay cannot drift apart. `SimAgent` and `SimBullet` are exported for the overlay's renderer.

`presimulate / presimulateAgainstGhost` — `game/ga/evolution.ts`

```
presimulate(generations: number): Population   // round-robin, every agent vs every
    other
presimulateAgainstGhost(
  generations:       number,
  ghost:             PlayerGhost,
  startPopulation:   Population,
  crossoverType:     'uniform' | 'single-point' = 'uniform',
  hallOfFameGhost?:  PlayerGhost,
  onLastEvaluation?: (individuals: Individual[]) => void,
): Population
```

`presimulate` pre-warms a population from scratch with a round-robin tournament (every agent fights every other) — opponent-agnostic training with no player involved. `presimulateAgainstGhost` is the one Solo actually uses: each generation scores every individual against the recording of the player's last round, so the agents adapt to *that* player's style.

Notes. A `hallOfFameGhost` adds a second evaluation against an earlier strong round; the two scores are averaged, which stops the population from over-fitting to the player's most recent habits alone. `onLastEvaluation` fires on the final generation with the individuals *and their real ghost fitnesses* — exactly the scores that decided the next opponent — before `evolve` replaces them with offspring. That snapshot is what feeds the training replay.

`Worker protocol` — `game/ga/evolution.worker.ts`

Presimulation is the one shooter workload heavy enough to move off the main thread. Unlike the map/maze workers, this one is stateless: one message in, one message out.

Direction	Type	Payload and meaning
in	`PRESIM`	`ghost`, `hofGhost?`, `population`, `generations`, `mutationRate`, `mutationStrength`, `crossoverType`, `realFitness`, `injectionDeviation` — runs `presimulateAgainstGhost` (skipped when `generations` is 0 or the recording is empty) and then one `evolve` seeded with `realFitness`.
out	`DONE`	`population: Population`, `evaluated?: Individual[]` — the evolved population, plus the last evaluated presim generation for the training replay (absent when no presim ran).
out	`ERROR`	`message: string`.

6.4 Stores (`game/`)

Module-level observable singletons built on `createListenable()` (Chapter 3): components read the fields directly and subscribe for changes. No Redux/Zustand.

`gameStore` — `game/gameStore.ts`

Holds the live `GameState` outside React: the `requestAnimationFrame` loop writes it every frame, `ShooterCanvas` and `DNADisplay` read it. Keeping it out of React state is deliberate — a 60 Hz `setState` would re-render the tree every frame.

`analyticsStore` — `game/analyticsStore.ts`

```
interface RoundRecord { round; hitsLanded; hitsReceived; bulletsFired;
                        accuracy; dodged; fitness; generation; bestFitness;
                        playerFrames: PlayerGhostFrame[];
                        agentFrames:  AgentGhostFrame[]; agentDna: number[] }
analyticsStore.push(record: RoundRecord, maxRounds = 20); analyticsStore.clear();
```

The `rounds: RoundRecord[]` ring buffer behind `/Analytics`; `push` trims to the newest `maxRounds` entries.

`raidbossStore` — `game/raidbossStore.ts`

The community population, backed by Firestore (the app's only server state — see `lib/firebase.ts`). Types `RaidbossIndividual`, `RaidbossDoc`, `RaidbossSlot`.

```
getRaidbossStatus():  Promise<RaidbossDoc | null>
claimRaidbossSlot():  Promise<RaidbossDoc>   // reserve an unevaluated individual
submitRaidbossFitness(index, fitness, claimedDoc): Promise<void>
consumePendingSlot(): RaidbossSlot | null
getRaidbossActive() / setRaidbossActive(v) / subscribeRaidbossActive
```

One player fights one claimed individual and submits its fitness; once the generation is fully evaluated it ticks over. The `raidbossActive` flag is a plain local listenable (not Firestore) telling the UI which lobby to return to.

trainingReplayStore — game/trainingReplayStore.ts

```
interface TrainingReplayUI {
  evaluated: Individual[];  // last evaluated presim generation, real fitnesses
  ranking:   number[];      // indices into 'evaluated', best fitness first
  focusIdx:  number;        // currently focused candidate
}
open(evaluated, focusIdx); setFocus(focusIdx); close(); requestClose()
```

While the training replay is open, the side panels show *its* state instead of the live game: the left bar the fitness ranking, the right panel the focused candidate's DNA. `open` computes the ranking itself. `requestClose` is a hook the overlay installs so outside chrome (the "← Game" button) can close it.

6.5 Horde mode (`horde/`)

A steady-state EA: agents spawn in waves and every death immediately breeds a replacement, so evolution is continuous rather than per-round.

updateHorde (worked example) — horde/hordeEngine.ts

```
type HordeEA = Pick<HordeSettings,
  'mutationRate' | 'mutationStrength' | 'crossoverType' | 'shootCooldown'>;

function updateHorde(
  state:        HordeGameState,
  dt:           number,
  input:        InputState,
  pStats:       PlayerStats,
  ea:           HordeEA,
  activeModIds: string[] = [],
): HordeGameState
```

The pure per-frame horde update, including the per-death mini-GA. Same contract as `gameLoop.update`: no React, no-op unless `phase === 'playing'`. `makeInitialState(pop, map)` builds the starting state and `resetHordeEngine()` clears module scratch state between runs (`getHordeShieldAngle()` exposes the shield angle to the renderer). `agentRadius(dna)` / `agentOpacity(dna)` derive an agent's look from its horde-only genes.

Horde gene layout — `horde/hordeDna.ts`

A horde genome extends the shared DNA rather than replacing it:

Range	Constant	Meaning
$0 \ldots$ `DNA_LENGTH` $- 1$	—	The 8 shared genes
`LOOP_GENE_START`	`= DNA_LENGTH`	First movement-loop gene
next `LOOP_STEPS` (4)	`LOOP_STEP_DURATION` $= 0.6\,$s	A repeating movement pattern: each gene is one step's turn, up to $\pm$`LOOP_MAX_ANGLE_RAD` (70°) via `loopOffsetRad(gene)`
`SIZE_GENE_INDEX`	`= LOOP_GENE_START + LOOP_STEPS`	Body size
`OPACITY_GENE_INDEX`	`= SIZE_GENE_INDEX + 1`	Body opacity

`HORDE_STARTER_DNA_LENGTH` = `OPACITY_GENE_INDEX` $+ 1$ is the total.

Notes. Because the layout is derived from `DNA_LENGTH`, adding a shared gene shifts the horde-only genes automatically. Tutorial presets: `HORDE_TUTORIAL_DNA` (inert dummies) and `HORDE_TUTORIAL_RAMP_DNA` (starter DNA with raised aggression — what the dummies switch to for the tutorial's "watch it evolve" half).

Maps, pathfinding, collision, rendering — `horde/`

```
// hordeMaps.ts
HORDE_MAPS: HordeMap[]; getHordeMap(id): HordeMap; CUSTOM_MAP_ID = 'custom'
buildCustomHordeMap(obstacles, spawnSides, playerSpawn): HordeMap
resolveHordeMap(mapId, customObstacles, customSpawnSides, customPlayerSpawn)
// hordePathfinding.ts
computeFlowField(obstacles, targetX, targetY): FlowField
sampleFlowField(field, x, y): { x, y } | null
// hordeCollision.ts
circleIntersectsObstacle(x, y, r, o): boolean
pushOutOfObstacles(x, y, r, obstacles)
bounceBulletOffObstacle(...)
// hordeRender.ts
render(ctx, state, shieldAngle = null);  HC = '#fb923c'  // horde accent colour
```

Pathfinding is a flow field: one BFS flood from the player's cell produces a direction per cell, which every agent then samples in $O(1)$ — one search per frame for the whole wave instead of one per agent. `hordeTypes.ts` holds the domain types (`HordePhase`, `HordeAgent`, `HordeObstacle`, `HordeSpawnSide`, `HordeMap`, `HordeGameState`). `hordeGameStore` holds the live `HordeGameState | null`; `hordeRunStore` keeps the last `HordeRunRecord` for the lobby overview.

6.6 Run mods / powerups (`mods/`)

`MOD_POOL / ModDefinition (worked example)` — `mods/modTypes.ts`

```
interface ModDefinition {
  id: string; name: string; description: string; icon: string;
  applyStats?:  (stats: PlayerStats) => PlayerStats;   // stat mod
  modifyShots?: (shots: ShotPlanEntry[]) => ShotPlanEntry[]; // behaviour mod
  repeatable?:  boolean;   // stackable up to MAX_MOD_STACKS
}
interface ShotPlanEntry { angleOffset: number; delay: number;
                          speedMult: number; homing?: boolean }
```

`MOD_POOL` holds the eight powerups offered during a run:

Mod	Kind	Effect
Speed Boost	stat, stackable	$\times 1.4$ move speed
Rapid Fire	stat, stackable	$\times 0.65$ shoot cooldown
Bullet Speed	stat, stackable	$\times 1.6$ bullet speed
Triple Shot	behaviour	3 bullets per shot, ± 0.18 rad spread
Burst Shot	behaviour	3 staggered shots ($+0.06$ s each) *and* $\times 3$ cooldown, so bullets/second stay constant
Homing Rounds	behaviour	Bullets curve toward the agent
Orbit Shield	engine	3 orbs circle the player (block bullets in Solo/Raidboss, destroy agents in Horde)
Ricochet Rounds	engine	Bullets bounce off arena walls once

Notes. Three kinds, not two: besides stat and behaviour mods, Orbit Shield and Ricochet Rounds declare *neither* callback — the engines (`gameLoop.ts` / `hordeEngine.ts`) check their ids (`SHIELD_MOD_ID`, `RICOCHET_MOD_ID`) directly. Burst Shot shows the two callbacks are not exclusive: it sets both. Only stat mods are `repeatable`; applying a `modifyShots` twice would be meaningless.

`applyMods / computeShotPlan` — `mods/modTypes.ts`

```
applyMods(base: PlayerStats, activeIds: string[]): PlayerStats
computeShotPlan(activeIds: string[]): ShotPlanEntry[]
modStackCount(activeIds, id): number
isModOfferable(mod, activeIds): boolean
stackOfferLabel(activeIds, id): string   // "Stack -> xN" | "Stackable"
MAX_MOD_STACKS = 5
```

`applyMods` folds every active stat mod over the base `PlayerStats`; because it runs once per *entry* in `activeIds`, stacking compounds for free. Every result is clamped to the same ranges as the settings sliders (move $50 \ldots 600$, bullet $100 \ldots 1000$, cooldown $0.05 \ldots 2$), so no stack can reach a broken value. `computeShotPlan` starts from a single shot and lets each behaviour mod transform the plan, hard-capped at `MAX_SHOTS_PER_TRIGGER` (12) so combinations cannot explode the $O(\text{bullets})$ collision pass. It is called only on an actual trigger pull, not per frame.

shotEngine / runModsStore — mods/

shotEngine.ts owns the special-shot behaviour and its tunables: steerHomingBullet(...) with HOMING_TURN_RATE (1.2 rad/s) and homingActive(lifetime) (HOMING_DURATION 0.5 s); tryWallBounce(...) with RICOCHET_MOD_ID / RICOCHET_BOUNCES (1); shieldBlocks(...) with SHIELD_MOD_ID, SHIELD_ORB_COUNT (3), SHIELD_ORBIT_RADIUS (55), SHIELD_ORB_RADIUS (10) and SHIELD_SPIN_RATE (2.6 rad/s); plus the QueuedShot type for delayed burst shots. runModsStore holds the active mod ids of the current run as a **multiset** — a stackable mod appears once per stack: addMod(id) (one stack, respects the cap), removeMod(id) (one copy), toggleMod(id) (on/off, for the settings grid), count(id), reset(). Listenable; it always assigns a fresh array.

6.7 Hooks (hooks/)

useGameLoop, useInput, useTouchControls — hooks/

useGameLoop({ ...}) runs the requestAnimationFrame loop: it calls update() each frame and writes the result into gameStore. useInput(): RefObject<InputState> merges keyboard and mouse into an input snapshot — a *ref*, not state, so input never triggers a re-render. useTouchControls(inputRef, canvasRef) adds the mobile joystick/aim zones and returns a RefObject<TouchVisualState> for drawing the joystick overlay.

useTutorialStep — hooks/useTutorialStep.ts

useTutorialStep<T extends string>(initial: T) — the coachmark state machine shared by ShooterCanvas and HordeCanvas: step / stepRef (current step), advance(next) (switch and lock detection), dismiss() (un-pause and open a 1.7 s grace window), graceOver(), and the loop-readable pausedRef / setPaused. The refs exist because the game loop reads them every frame from outside React.

6.8 Components (components/)

ShooterCanvas / HordeCanvas — components/

```
ShooterCanvas({ scale?, externalInputRef?, leaveHandlerRef?,
                tutorial?, tutorialMode? })   // 'solo' | 'raidboss'
HordeCanvas({ scale?, externalInputRef?, touchControls?, tutorial? })
```

The two game canvases: each owns the loop wiring, the overlays, and the round lifecycle, and reads its store (gameStore / hordeGameStore). externalInputRef switches to the page-supplied zone-touch input; leaveHandlerRef lets the page register a clean-exit callback. tutorial runs the practice round — for the shooter a passive target and a single round, with coachmark steps (move → aim → shoot → done); tutorialMode only selects which explainer follows and which lobby to return to. Horde's practice round uses a small wave (TUTORIAL_WAVE_SIZE = 8) and never persists to hordeGameStore.

EvolutionReplayOverlay — components/EvolutionReplayOverlay.tsx

EvolutionReplayOverlay({ ghost, evaluated, onClose }) — the training replay opened from the DNA reveal. Every candidate of the last presim generation plays live against the player's recorded round (through createGhostSim, so it is the very fight that was scored); one focused agent is drawn in full detail, and a side panel ranks the real fitnesses with the top ELITE_COUNT marked as elites. It answers "why am I facing *this* DNA next?". Only offered when a presim actually ran.

DNADisplay, HordeDnaPanel, touch zones — components/

DNADisplay() shows the current opponent's DNA and subscribes to gameStore. HordeDnaPanel({ bestDna, height }) is the "Best DNA" side panel (width PANEL_W = 200). MobileJoystickZone and MobileAimZone are the touch input zones.

Tutorial visuals — components/

arenaAgentSim.ts is the shared small-arena physics behind every tutorial preview canvas (ARENA_SIZE = 240 px, ARENA_SCALE = 240/800 — the real GAME_CONFIG constants scaled down, since updateAgent's formulas assume an 800 px arena): stepArenaAgent(...), drawArenaAgentTriangle, patrol, lineupSpots, drawGenLabel, bulletHits, clamp01 and the ArenaAgentState / ArenaTarget / ArenaBullet types. On top of it: DnaPreviewCanvas({ dna }) / HordeDnaPreviewCanvas({ dna }) (live agent preview driven by the gene sliders), GhostArenaVisual({ variant?, count? }) ('replay' | 'lineup' | 'selection' | 'nextgen'), FitnessArenaVisual({ agentColor?, agentLabel? }), RaidbossArenaVisual({ variant, count, startGen? }) ('community' | 'evaluate' | 'generation') and HordeArenaVisual({ variant }) ('fitness' | 'evolution' | 'elites'). The explainer flows themselves live in tutorialEvolutionContent.tsx (TutorialEvolutionExplainer, SoloDnaExplainerHint, SoloEaSettingHint), tutorialRaidbossContent.tsx (TutorialRaidbossExplainer) and tutorialHordeContent.tsx (TutorialHordeExplainer).

6.9 Lobby and settings

Lobby — lobby/

ShooterLobbyPage (default export) is the root: the mode picker plus the three lobbies NormalLobby, RaidbossLobby and HordeLobby({ initialTab? }), each with its own tabs and overview (SoloPlayOverview, HordeOverview). Every Tutorial button consults hasCompletedAnyTutorial(): returning players get TutorialChooserModal({ onPractice, onExplainer, onClose, accent }), brand-new players go straight into the practice round. Supporting modules: lobbyConstants.ts (LobbyMode, SHOOTER_MODES, PRESETS, HORDE_PRESETS, the tab definitions, and the horde gene descriptors HORDE_BAR_GENES / HORDE_LOOP_GENES / HORDE_EDITABLE_GENES), lobbyHooks.ts (useMobile, useZoom, enterGameFullscreen), lobbyStyles.ts, TopBar, DnaGeneRow (read-only gene bar) and previews/ (the animated mode previews ShooterPreview, RaidbossPreview, HordePreview, HordeMapPreview).

ShooterSettingsPanel — settings/ShooterSettings.tsx

ShooterSettingsPanel plus the reusable sections it is built from: ShooterDnaSection({ dna, onChange, onBeforeChange?, genes? }) — the gene sliders, driven by a DnaGeneDescriptor[] so the same component serves the shared genes, the horde-only genes and the tutorial explainers' live previews — ShooterPlayerSection, ShooterRoundSection({ onBeforeChange?, locked? }) and HordeWaveSection.

Chapter 7

Data Formats

Serialization formats are the closest thing the app has to a public API: they cross device boundaries via share codes and local storage.

7.1 Peak Finder map code (v1)

Produced by `encodeMap` (`BattleShips/engine/mapCodec.ts`): a URL-safe Base64 encoding of

```
{ "v": 1,                       // format version
  "id": "ABC123",               // map id
  "m": [[x, y, isGlobal, floor?], ...],  // minima (normalized coords)
  "wr": 0.04,                   // win radius (fraction of map width)
  "bs": 0.12,                   // basinScale (optional; see below)
  "t": 1699999999999 }          // timestamp
```

The optional 4th tuple entry (`floor`) is only present for local minima with an explicit depth — omitting it keeps older codes valid and round-trips untouched (random-floor) nodes unchanged; the stored floor is a fraction of `basinScale`. The optional `bs` (basin scale) is written only when present on the config; older codes without it decode with the Medium preset's `basinScale`, so they stay valid. Coordinates are rounded to 4 decimals. `decodeMap` throws a single `Error('Invalid map code')` for anything unparsable or with an unknown `v`; a missing `wr` defaults to 0.04 and a missing `t` to `Date.now()`.

7.2 Maze code

Produced by `encodeMaze` in `mazeGame/engine/mazeCodec.ts`: a URL-safe Base64 encoding of

```
{ "v": 1,             // format version
  "id": "ABC123",     // content-derived id (djb2 hash, base36)
  "c": cols, "r": rows,
  "s": [x, y],        // start cell
  "g": [x, y],        // goal cell
  "w": "af3...",      // one hex digit per cell: open-passage nibble
                      // (N=1 E=2 S=4 W=8), row-major
  "t": 1699999999999 }// timestamp
```

The full wall grid is encoded (not a generator seed), so hand-built creator mazes round-trip exactly; 32×32 — the creator's maximum — is 1024 hex digits, comfortably copy-pasteable. `decodeMaze` validates version, dimensions (2..64), wall-string length and marker bounds, throwing `Error('Invalid maze code')` otherwise.

7.3 Function problem code

Produced by `encodeFunctionCode` in `BattleShips/engine/functionProblem.ts`: a URL-safe Base64 encoding of

```
{ "k": "fn",              // marker - distinguishes from map codes
  "fn": "rastrigin",      // benchmark id, "procedural", or "random"
  "c": [cx, cy],          // optimum position in the [0,1] viewport
  "th": theta,            // rotation (radians)
  "s": [sx, sy],          // anisotropic scale
  "r": [rx, ry],          // reflections (1 | -1)
  "sd": seed }            // only for procedural surfaces
```

`decodeProblem` (`BattleShips/engine/problemCode.ts`) tries the function decoder first (keyed on the `k:"fn"` marker), then falls back to map decoding. A code with `fn:"random"` is a sentinel that resolves to a fresh procedural surface on every load.

7.4 Local storage keys

Key	Contents
`app.hints.enabled`	Hint system on/off (`localStorage`)
`app.hints.seen`	Seen hint ids (`sessionStorage`, per session)
`bs.savedMaps`	Saved Peak Finder maps (`useSavedMaps`)
`bs.lastCopiedCode`	In-app clipboard fallback for share codes
`maze.savedMazes`	Saved mazes (`useSavedMazes`)
`shooter.tutorial.completed`	Solo tutorial finished
`horde.tutorial.completed`	Horde tutorial finished
`raidboss.tutorial.completed`	Raidboss tutorial finished

Appendix A

Extension Recipes

Short how-tos for the most likely extension points.

Add a new analytic problem to Peak Finder
Add the closed-form function to `BENCHMARK_FUNCTIONS` (`engine/functionProblem.ts`) and list its id under the right entry of `FUNCTION_CATEGORIES`. Nothing else is required: `createFunctionProblem` handles the affine transform, the grid-sampled normalisation and the win target, and `SavedFunctionsSidebar` picks the function up from the registry. Give the function a sharpening exponent only if its raw range makes the surface unreadable — and shape `evaluate`, never the display curve.

Add a new EA operator
Add the implementation to the relevant registry in `engine/ea/operators.ts` and add its id to the matching `*Strategy` union in `types/ea.ts` — the union is what makes the settings panel offer it. For crossover, implement the *recording* variant (`CROSSOVER_STRATEGIES_RECORDING`) first and let the plain variant wrap it, so the replay keeps working; the recording variant must return every random choice it made.

Add a new maze fitness function
Extend `FitnessFnId` (`mazeGame/types/maze.ts`), add the id and label to `FITNESS_FN_IDS` / `FITNESS_FN_LABELS`, and implement the scorer in `mazeGame/engine/mazeProblem.ts`. Keep the contract: lower = better, roughly normalized to $[0, 1]$, and scored on the walk's *final* cell. If the score depends on the whole population (as novelty does), return a constant from `evaluate` and apply a scorer in the stepper instead.

Add a new shooter gene
Add the key to `DNA_INDEX` (`shooterGame/shooter.types.ts`) — `DNA_LENGTH`, the horde gene offsets and the crossover/mutation loops all derive from it. Then add a `DNA_GENE_INFO` entry (label + tooltip), extend `STARTER_DNA` / `TUTORIAL_DNA` to the new length, and consume the gene in `updateAgent` (`game/core/gameLoop.ts`). Genes stay in $[0, 1]$; map to real units at the point of use.

Add a run mod / powerup
One entry in `MOD_POOL` (`shooterGame/mods/modTypes.ts`). Use `applyStats` for a stat multiplier (add `repeatable: true` to make it stackable — it compounds by itself) and/or `modifyShots` to reshape the shot plan. A mod that needs real engine behaviour declares neither and is checked by id in the engines, as Orbit Shield and Ricochet Rounds are.

Add a help topic
One entry in `HELP_TOPICS` (`components/help/helpContent.tsx`) with its `Gameplay` and `Technical` tab components, plus a `<HelpButton topic="..."/>` where it should appear.

Add a hint
One entry in `components/hints/hintContent.ts` (or `mazeGame/hints/mazeHintContent.ts`) — the wording lives there, never at the call site. Fire it with `showHint(id)`, or wrap the anchor element in `<HintPopover id="..."/>` for a coach-mark.

Add a new mini-game module
Follow the four-layer structure of Chapter 3 (`types/` → `engine/` → `hooks/` → `components/`, never importing upward), run any EA in a worker behind a typed `WorkerInMessage`/`WorkerOutMessage` union, reuse `useReplayPlayer` and `useSavedLibrary` rather than re-implementing them, and register the route in `App.tsx`.

Appendix B

Tunable Constants Reference

The constants worth turning first, and where they live. Defaults are the values in the source at the time of writing; each entry's own chapter explains what it does in context.

Constant	Default	Effect
Peak Finder — `modules/BattleShips/`		
engine/functionSurface.ts		
`FAR_FALLOFF`	1.3	How steeply open ground climbs outside a basin. At 1.0 it never gets past orange; much above 1.3 it saturates to red right off the rim.
`SMOOTH_K`	0.12	Widest smooth-min blend merging the cones (0 = hard creases at every Voronoi seam). Capped per map against its tightest minima gap.
`LOCAL_MIN_FLOOR_MIN/MAX`	0.18 / 0.55	Local minimum depth, as a *fraction of* `basinScale` — equally deceptive at every map size.
engine/mapCodec.ts, engine/colorScale.ts, types/ea.ts		
`MAP_SIZES`	see below	Size presets: minima count, win radius, min spacing, `basinScale`.
`STOPS`	see file	The colour ramp (violet = summit → red = far).
`DEFAULT_EA_CONFIG`	Ch. 4	Vs-EA defaults (= the `'normal'` preset).
engine/ea/, components/game/		
`WIN_POPULATION_FRACTION`	0.10	In `evolutionaryAlgorithm.ts`. Fallback only — `EAConfig.winPopulationFraction` (0.35) overrides it.
`DEFAULT_HEATMAP_CONFIG`	Ch. 4	In `HeatmapLayer.tsx`: `resolution` 1080, `opacity` 0.82, `revealRadius` 0.05, `valueExponent` 1.
`DOT_MOVE_DURATION_MS`	1000	In `ReplayMap.tsx`. Replay dot glide duration.
Maze Explorer — `modules/mazeGame/`		
engine/mazeProblem.ts		
`DEFAULT_BRAID`	0.5	Fraction of dead-ends opened into loops.

continued on the next page

Constant	Default	Effect
DEFAULT_OPENNESS	0.25	Fraction of interior walls removed. Too high and Manhattan stops being deceptive — which is the whole demo.
MAX_PATH_LENGTH	600	Hard cap on genome length.
engine/ea/novelty.ts, types/ea.ts		
DEFAULT_K	15	Neighbours averaged for the novelty score.
DEFAULT_MAZE_EA_CONFIG	Ch. 5	Maze EA defaults.
Shooter — modules/shooterGame/		
shooter.types.ts		
GAME_CONFIG	Ch. 6	Round duration, population, elites, mutation, speeds, cooldowns, radii.
ARENA	800×800	Logical canvas size.
game/ga/, mods/, horde/		
INJECTION_COUNT	4	In evolution.ts. Jittered copies of the best DNA injected each generation — the anti-convergence valve.
MAX_MOD_STACKS	5	In modTypes.ts. Stack cap per mod.
MAX_SHOTS_PER_TRIGGER	12	In modTypes.ts. Hard cap so mod combinations cannot explode bullet collision.
HOMING_TURN_RATE	1.2	In shotEngine.ts. rad/s (HOMING_DURATION 0.5 s).
SHIELD_*	3 / 55 / 10 / 2.6	In shotEngine.ts. Orb count, orbit radius, orb radius, spin rate.
LOOP_STEPS	4	In hordeDna.ts. Movement-loop genes (LOOP_STEP_DURATION 0.6 s, LOOP_MAX_ANGLE_RAD 70°).
Shared		
autoplayIntervalMs	1200	In hooks/useReplayPlayer.ts. Dwell per replay frame (the maze overlay passes 1600).
LAYOUT	see file	In components/layout/GameLayout.tsx: SPACING 16, LEFT_BAR 240, RIGHT_PANEL 220 — reference widths; the grid clamps the real ones.

The Peak Finder size presets (MAP_SIZES) in full:

Size	Minima	Win radius	Min spacing	basinScale
small	4–6	0.05	0.20	0.18
medium	8–11	0.04	0.13	0.12 (DEFAULT_MAP_SIZE; also the fallback for legacy codes without bs)
large	13–17	0.03	0.10	0.09
huge	20–26	0.022	0.075	0.068